Norme di Progetto

QuantSys Institutional

May 25, 2026

Contents

1 Processo di supporto **2**

1.1 Repository . . . 2

1.1.1 Repository GitHub . . . 2

1.1.2 Struttura della repository . . . 2

1.1.3 Nomenclatura dei branch . . . 3

1.1.4 Cache dei dati . . . 4

1.2 Piano di documentazione . . . 4

1.2.1 Documenti accademici (cartella `PB/`) . . . 4

1.2.2 Documenti operativi (radice repository) . . . 4

1.2.3 Documentazione interna alle strategie . . . 5

1.2.4 Aggiornamento e versionamento . . . 5

1.3 Strumenti . . . 5

1.3.1 Linguaggio e runtime . . . 5

1.3.2 Provider dati . . . 6

1.3.3 Strumenti di collaborazione . . . 6

1.3.4 Strumenti di documentazione . . . 6

2 Processo di sviluppo **7**

2.1 Obiettivo . . . 7

2.2 Flusso operativo generale . . . 7

2.3 Procedura Git . . . 8

2.3.1 Setup iniziale - Clonare la repository . . . 8

2.3.2 Creazione di una nuova feature . . . 8

2.3.3 Caricamento dei cambiamenti . . . 9

2.3.4 Sincronizzazione con `main` . . . 10

2.4 Pull Request . . . 10

2.4.1 Apertura della Pull Request . . . 11

2.4.2 Template di descrizione . . . 11

2.4.3 Code review . . . 12

2.4.4 Merge della Pull Request . . . 12

2.5 Comandi git essenziali . . . 13
2.6 Best practices . . . 14
2.6.1 Linee guida sui commit . . . 14
2.6.2 Linee guida sui branch . . . 14
2.6.3 Linee guida sulle pull request . . . 15
2.6.4 Linee guida sul codice . . . 15
2.6.5 Anti-pattern da evitare . . . 15

3 Sviluppo di strategie di trading 16
3.1 Obiettivo . . . 16
3.1.1 Razionale della procedura . . . 16
3.2 Dalla lettura della issue alla creazione del branch . . . 17
3.2.1 Step 1: Identificazione della issue . . . 17
3.2.2 Step 2: Lettura della documentazione tecnica . . . 17
3.2.3 Step 3: Aggiornamento del branch `main` locale . . . 17
3.2.4 Step 4: Creazione del branch dedicato . . . 18
3.3 Implementazione della strategia . . . 18
3.3.1 Step 5: Creazione della cartella della strategia . . . 18
3.3.2 Step 6: Implementazione del codice . . . 18
3.3.3 Step 7: Configurazione del DESCRIPTOR . . . 19
3.3.4 Step 8: Creazione dello script walk-forward . . . 19
3.3.5 Step 9: Commit incrementali . . . 20
3.4 Validazione walk-forward . . . 20
3.4.1 Step 10: Lancio del walk-forward . . . 20
3.4.2 Step 11: Analisi dei risultati . . . 20
3.4.3 Step 12: Decisione di accettazione o scarto . . . 21
3.5 Esito positivo: integrazione tramite Pull Request . . . 22
3.5.1 Step 13a: Aggiornamento documentazione . . . 22
3.5.2 Step 14a: Pull request finale . . . 22
3.5.3 Step 15a: Review e merge . . . 23
3.5.4 Step 16a: Cleanup . . . 24
3.6 Esito negativo: archiviazione con disabilitazione del discovery . . . 24
3.6.1 Convenzione del prefisso underscore . . . 24
3.6.2 Step 13b: Rinomina della cartella della strategia . . . 24
3.6.3 Step 14b: Creazione del file `_FAILED.md` . . . 25
3.6.4 Step 15b: Aggiornamento documentazione globale . . . 26
3.6.5 Step 16b: Pull request di archiviazione . . . 26
3.6.6 Step 17b: Cleanup . . . 27
3.6.7 Note operative sul `_FAILED.md` . . . 27

3.7 Quadro riassuntivo 28

4 Verbali e meeting **29**

4.1 Obiettivo 29

4.2 Struttura del verbale 29

4.3 Procedura 29

4.3.1 Frequenza 30

Capitolo 1 Processo di supporto

1.1 Repository

Questa sezione descrive la repository del progetto QuantSys Institutional, la sua struttura e le modalità di organizzazione del codice e della documentazione.

1.1.1 Repository GitHub

Il progetto QuantSys Institutional è ospitato su una singola repository GitHub:

- **Repository**: https://github.com/Idrorepellente/quantsys_institutional

A differenza di progetti che separano codice e documentazione in repository distinte, QuantSys Institutional adotta una repository monolitica in cui codice, documentazione e file di configurazione coesistono. Questa scelta è motivata dal fatto che molti documenti (es. `GUIDE_STRATEGIE_CUSTOM.md`, `STRATEGIE_FX_CANDIDATE.md`) sono strettamente accoppiati al codice e devono evolvere assieme ad esso.

1.1.2 Struttura della repository

La repository è organizzata in cartelle tematiche, ognuna con una responsabilità ben definita:

- `core/`: tipi base, portfolio, allocator, risk manager, exit policy, indicator pool;
- `backtest/`: engine di backtest event-driven con check intra-bar;
- `data/`: universe, loader yfinance, loader Polygon, loader Dukascopy FX;
- `strategies/`: plugin di strategie atomiche (`momentum`, `quality_lv`, `value`, `custom_example`);
- `tests/`: script di walk-forward per ogni strategia, test unitari;
- `scripts/`: script di pipeline (`build_feature_store`, `run_ml_pipeline`, `bear_overlay_backtest` `download_polygon_m5`, `download_dukascopy_fx`);

- `ml/`: pipeline ML con feature engineering e walk-forward training;
- `dashboard/`: Flask app con UI Plotly, API JSON, runner backtest;
- `reporting/`: metriche, factor attribution, ensemble engine;
- `validation/`: walk-forward harness;
- `config/`: file YAML di configurazione (`quantsys.yaml`);
- `output/`: output centralizzati prodotti dai backtest (`strategies/`, `ml/`);
- `data/cache_d1/`, `cache_polygon_m5/`, `cache_fx_d1/`, `cache_fx_m5/`: cache di dati di mercato.

La radice della repository contiene inoltre i documenti Markdown principali:

- `README.md`: descrizione del progetto;
- `OBIETTIVI.md`: obiettivi misurabili del progetto;
- `GUIDE_STRATEGIE_CUSTOM.md`: guida per implementare nuove strategie nel framework plugin;
- `PITCH_FOF.md`: pitch deck verso Fund of Funds;
- `ROADMAP_FASE2_v2.md`: piano per Fase 2 (ampliamento ensemble);
- `STRATEGIE_FX_CANDIDATE.md`: proposta di strategie FX da implementare.

1.1.3 Nomenclatura dei branch

Il sistema di versionamento prevede la seguente convenzione di naming per i branch:

- `main`: branch principale, sempre in stato deployabile;
- `feature/<nome-strategia>`: per lo sviluppo di una nuova strategia plugin (es. `feature/stat-arb-coint`);
- `feature/<numero-issue>-<descrizione>`: per feature non legate a strategie (es. `feature/12-exit-policy-engine`);
- `fix/<numero-issue>-<descrizione>`: per bug fix (es. `fix/27-stale-feature-store`);
- `docs/<argomento>`: per modifiche esclusivamente documentali (es. `docs/exit-policy-guide`).

Tutti i branch devono essere collegati ad almeno una issue GitHub, che ne motiva l'esistenza e ne traccia lo scopo.

1.1.4 Cache dei dati

Le cartelle `data/cache_*` contengono dati di mercato scaricati da provider esterni (Polygon, yfinance, Dukascopy) e **non vengono versionate su GitHub** (sono escluse via `.gitignore`). Ogni sviluppatore che clona la repository deve eseguire gli script di download per popolare le cache.

Analogamente, la cartella `output/` contiene risultati di backtest specifici dell'ambiente di esecuzione e non viene versionata.

1.2 Piano di documentazione

Il piano di documentazione definisce il contenuto e la struttura dei documenti del progetto QuantSys Institutional.

1.2.1 Documenti accademici (cartella `PB/`)

Questi documenti sono prodotti in LaTeX e versionati nella cartella `PB/` della repository:

- **Analisi dei requisiti** (`analisi_dei_requisiti/`): descrive i requisiti funzionali e non funzionali del sistema, i casi d'uso, le interfacce e la tracciabilità;
- **Manuale utente** (`manuale_utente/`): guida all'installazione, configurazione, avvio e troubleshooting del sistema;
- **Norme di progetto** (`norme_di_progetto/`): documento corrente. Definisce processi, convenzioni, responsabilità e best practices;
- **Specifica tecnica** (`specifica_tecnica/`): descrive l'architettura del sistema, i moduli principali, le interfacce interne e le scelte tecnologiche;
- **Verbali** (`verbali/`): resoconti degli incontri di team e delle decisioni prese.

1.2.2 Documenti operativi (radice repository)

Questi sono documenti Markdown che evolvono con il codice e descrivono aspetti operativi e di sviluppo:

- `README.md`: punto di ingresso per nuovi sviluppatori. Contiene descrizione, quick-start e link agli altri documenti;
- `OBIETTIVI.md`: obiettivi misurabili del sistema (CAGR netto, Sharpe target, alpha vs benchmark, capacity);

- `GUIDE_STRATEGIE_CUSTOM.md`: guida tecnica per sviluppatori che vogliono implementare una nuova strategia nel framework plugin. Documento esaustivo che copre struttura cartella, descriptor, indicator pool, universe filtering, exit policy, test e troubleshooting;
- `PITCH_FOF.md`: documento di pitch verso Fund of Funds, costruito a partire dai numeri OOS validati;
- `ROADMAP_FASE2_v2.md`: piano per l'ampliamento dell'ensemble con nuove strategie;
- `STRATEGIE_FX_CANDIDATE.md`: proposta di sei strategie FX scrivibili nel framework plugin, da validare in walk-forward.

1.2.3 Documentazione interna alle strategie

Ogni strategia plugin (es. `strategies/momentum/`) può contenere al proprio interno:

- `README.md`: razionale economico, riferimenti letteratura, parametri default, risultati OOS;
- File Markdown con prefisso `_` (es. `_FAILED.md`, `_NOTE.md`): documenti interni che il sistema di discovery ignora, utilizzati per registrare tentativi falliti o note di lavoro. La convenzione del prefisso `_` è descritta in dettaglio nel Capitolo 3.

1.2.4 Aggiornamento e versionamento

Ogni documento accademico deve essere aggiornato regolarmente. Le modifiche ai documenti sono effettuate tramite pull request, esattamente come per il codice, in modo da garantire revisione e tracciabilità.

I documenti Markdown operativi (radice repository) evolvono naturalmente con il codice e vengono aggiornati nelle stesse pull request che modificano le funzionalità correlate.

1.3 Strumenti

Questa sezione elenca gli strumenti utilizzati nello sviluppo del progetto.

1.3.1 Linguaggio e runtime

- **Python 3.11+**: linguaggio principale dell'intero progetto;
- **Virtual environment** (`venv`): isolamento delle dipendenze. Tutte le dipendenze sono dichiarate in `requirements.txt`;

- **Pandas**, **NumPy**, **SciPy**: manipolazione dati e calcoli numerici;
- **scikit-learn**: ML pipeline (Random Forest, gradient boosting);
- **statsmodels**: regressioni multi-factor, cointegration test;
- **Flask**: dashboard web di monitoring;
- **Plotly**: visualizzazioni interattive nella dashboard;
- **PyYAML**: parsing dei file di configurazione.

1.3.2 Provider dati

- **Polygon.io**: dati equity D1 e M5 per S&P 500 (tier Stocks);
- **Dukascopy**: dati FX D1 e M5 per major pair (via package `dukascopy-python`);
- **yfinance**: fallback gratuito per dati D1 storici.

1.3.3 Strumenti di collaborazione

- **GitHub**: versionamento codice, issue tracking, pull request, code review;
- **GitHub Issues**: registrazione di feature da implementare, bug, decisioni di design da prendere;
- **GitHub Projects**: board Kanban con stati *Todo*, *In Progress*, *Review*, *Done*;
- **Notion**: collaborazione informale, brainstorming pre-issue, documentazione di lavoro non ancora consolidata.

1.3.4 Strumenti di documentazione

- **LaTeX**: documenti accademici della cartella `PB/`;
- **Markdown**: documenti operativi della radice repository;
- **Mermaid**: diagrammi inseriti in documenti Markdown quando utili.

Capitolo 2 Processo di sviluppo

2.1 Obiettivo

In questo capitolo viene descritto il flusso operativo completo per lo sviluppo di nuove feature e per la pubblicazione delle modifiche sulla repository QuantSys Institutional. Il flusso copre tutti i passi: dalla creazione di una issue GitHub, attraverso la creazione del branch dedicato e lo sviluppo locale, fino alla pull request e al merge finale.

Per il caso specifico dello sviluppo di nuove strategie di trading, che ha alcune particolarità (validazione walk-forward, esito positivo o negativo, exclusione da discovery), si rimanda al Capitolo 3.

2.2 Flusso operativo generale

Il ciclo di sviluppo standard per una qualsiasi modifica al progetto QuantSys Institutional si articola nelle seguenti fasi:

1. **Apertura issue**: ogni modifica deve essere preceduta dalla creazione di una issue GitHub che descrive scopo, motivazione e criteri di accettazione;
2. **Clone (o sincronizzazione) della repository**: se non già fatto, clonare il repository localmente. Altrimenti aggiornare il branch `main` locale;
3. **Creazione del branch dedicato**: a partire da `main` aggiornato, creare un branch nominato secondo la convenzione descritta nella Sezione 1;
4. **Sviluppo locale**: scrivere il codice, eseguire i test locali (unitari e di integrazione), aggiornare la documentazione se necessario;
5. **Commit incrementali**: effettuare commit piccoli, atomici e con messaggio descrittivo;
6. **Push del branch**: pubblicare il branch sul remoto GitHub;
7. **Apertura pull request**: aprire una pull request verso `main` collegata alla issue di partenza;

8. **Code review**: attendere la revisione da parte di almeno un altro membro del team;
9. **Eventuali modifiche**: implementare le modifiche richieste in review;
10. **Merge**: una volta approvata, effettuare il merge della pull request;
11. **Chiusura issue**: la issue viene chiusa automaticamente se nel testo della PR è presente la formula `Closes #<numero>`.

2.3 Procedura Git

2.3.1 Setup iniziale - Clonare la repository

Per iniziare a lavorare al progetto è necessario clonare la repository. Eseguire dal terminale:

```
git clone https://github.com/Idrorepellente/quantsys_institutional.git
cd quantsys_institutional
```

Successivamente, creare un virtual environment e installare le dipendenze:

```
python -m venv venv
# Su Windows PowerShell:
.\venv\Scripts\Activate.ps1
# Su Linux/macOS:
source venv/bin/activate

pip install -r requirements.txt
```

Per i dettagli completi sul setup dell'ambiente (configurazione `.env`, scaricamento delle cache dati) si rimanda al Manuale Utente.

2.3.2 Creazione di una nuova feature

Il flusso operativo per la creazione di una nuova feature si articola nei seguenti step.

Step 1: Identificare o creare la issue

Ogni nuova feature deve essere collegata a una issue su GitHub. Se non esiste già, aprire una nuova issue specificando:

- **Titolo**: descrittivo e conciso;
- **Descrizione**: motivazione della feature, criteri di accettazione, eventuali riferimenti a letteratura o discussioni precedenti;
- **Labels**: assegnare label appropriati (es. `enhancement`, `bug`, `strategy`, `docs`);

- **Assignee**: indicare il responsabile dello sviluppo.

Annotare il numero della issue (es. #42), che verrà utilizzato per il nome del branch.

Step 2: Aggiornare il branch `main` locale

Prima di creare il nuovo branch, assicurarsi di avere l'ultima versione di `main`:

```
git checkout main
git pull origin main
```

Step 3: Creare il branch dedicato

Creare il branch utilizzando la convenzione di naming descritta nel Capitolo 1:

```
git checkout -b feature/42-nome-descrittivo
```

Step 4: Sviluppo locale

Effettuare le modifiche necessarie. Durante lo sviluppo è opportuno effettuare commit incrementali, ciascuno con un messaggio chiaro:

```
git add .
git commit -m "Aggiungi calcolo ATR(14) all'indicator pool"
```

In fase di sviluppo, se si tocca codice rilevante per la pipeline di backtest, è raccomandato:

- Eseguire i test unitari della cartella `tests/`;
- Effettuare almeno un walk-forward completo della strategia o componente coinvolto, verificando che i numeri OOS non degradino rispetto allo stato precedente.

2.3.3 Caricamento dei cambiamenti

Step 1: Verificare lo stato dei file

Prima di eseguire il push, verificare lo stato del working directory:

```
git status
```

Step 2: Aggiungere i file modificati

```
git add .
```

Se si desidera aggiungere solo specifici file:

```
git add core/exit_policy.py strategies/momentum/strategy.py
```

Step 3: Eseguire il commit

```
git commit -m "Aggiungi exit_policy_type al StrategyDescriptor"
```

Step 4: Eseguire il push

```
git push origin feature/42-nome-descrittivo
```

Al primo push di un branch nuovo è necessario specificare l'upstream:

```
git push --set-upstream origin feature/42-nome-descrittivo
```

2.3.4 Sincronizzazione con `main`

Se il branch di lavoro è "in vita" da molti giorni, è opportuno periodicamente sincronizzarlo con `main` per evitare conflitti al merge:

```
git checkout main
git pull origin main
git checkout feature/42-nome-descrittivo
git merge main
```

In caso di conflitti:

1. Aprire i file con conflitti nell'editor (sono evidenziati da `git status`);
2. Risolvere manualmente i conflitti scegliendo cosa mantenere;
3. Eseguire `git add <file>` sui file risolti;
4. Concludere il merge con `git commit`;
5. Effettuare il push.

2.4 Pull Request

La pull request è il meccanismo formale con cui si propone l'integrazione delle modifiche nel branch `main`. Tutte le modifiche, anche quelle apparentemente piccole, devono passare per una pull request: non sono ammessi push diretti su `main`.

2.4.1 Apertura della Pull Request

Step 1: Accedere alla repository su GitHub

Navigare a `https://github.com/Idrorepellente/quantsys_institutional`. Dopo aver eseguito il push del branch, GitHub mostra automaticamente un pulsante **Compare & pull request**.

In alternativa, è possibile aprire manualmente la pull request:

1. Navigare nella tab **Pull requests**;
2. Cliccare su **New pull request**;
3. Selezionare `main` come *base* e il proprio branch come *compare*;
4. Cliccare su **Create pull request**.

Step 2: Compilare i dettagli della Pull Request

Una buona pull request comunica chiaramente cosa è stato cambiato, perché, e come testarlo.

Titolo Deve essere descrittivo e conciso. Esempi:

- Buono: *Aggiungi exit policy ATR-based al framework strategie*
- Cattivo: *Fix*, *Updates*, *Changes*

Descrizione Deve includere:

- Riferimento alla issue chiusa: `Closes #<numero>`;
- Descrizione sintetica dei cambiamenti effettuati;
- Eventuali note importanti (breaking change, dipendenze nuove, test da eseguire manualmente);
- Per modifiche al codice di backtest o alle strategie, includere i numeri OOS pre/post modifica.

2.4.2 Template di descrizione

Si raccomanda l'uso del seguente template per le pull request:

```
Closes #<numero-issue>

## Descrizione
<descrizione sintetica dei cambiamenti>

## Cambiamenti principali
- <punto 1>
- <punto 2>
- <punto 3>

## Test eseguiti
- [ ] Test unitari della cartella tests/
- [ ] Walk-forward della strategia X (Sharpe pre: 0.81, post: 0.83)
- [ ] Verifica che la dashboard si avvii correttamente

## Note
<eventuali note: breaking changes, dipendenze nuove,
manual steps richiesti per applicare la modifica>
```

2.4.3 Code review

Una volta aperta la pull request:

1. Almeno un altro membro del team deve effettuare la review;
2. Il reviewer può:
    - **Approve**: la PR è pronta per il merge;
    - **Request changes**: sono richieste modifiche prima del merge;
    - **Comment**: feedback non bloccante.
3. L'autore della PR deve rispondere ai commenti, applicare le modifiche richieste, e successivamente effettuare nuovo push del branch (la PR si aggiorna automaticamente);
4. Una volta ottenuta l'approvazione, la PR può essere mergiata.

2.4.4 Merge della Pull Request

Esistono tre strategie di merge supportate da GitHub:

- **Merge commit**: mantiene la storia di tutti i commit del branch;

- **Squash and merge**: condensa tutti i commit del branch in un singolo commit su `main`;
- **Rebase and merge**: applica i commit del branch in cima a `main` senza commit di merge.

Per il progetto QuantSys Institutional si raccomanda **Squash and merge** per la maggior parte delle pull request, in modo da mantenere la storia di `main` pulita e leggibile.

Step finale: cancellare il branch

Dopo il merge, GitHub propone di cancellare il branch remoto. Si consiglia di farlo per mantenere la lista dei branch ordinata. Anche il branch locale può essere cancellato:

```
git checkout main
git pull origin main
git branch -d feature/42-nome-descrittivo
```

2.5 Comandi git essenziali

Di seguito i comandi principali utilizzati nel flusso di lavoro:

- `git clone <url>`: clonare una repository;
- `git status`: visualizzare lo stato dei file modificati;
- `git checkout <branch>`: cambiare branch;
- `git checkout -b <branch>`: creare e passare a un nuovo branch;
- `git add .`: aggiungere tutti i file modificati allo stage;
- `git add <file>`: aggiungere un file specifico;
- `git commit -m "messaggio"`: creare un commit con il messaggio specificato;
- `git push origin <branch>`: inviare i cambiamenti al repository remoto;
- `git push -set-upstream origin <branch>`: primo push di un branch nuovo (collega upstream);
- `git pull origin <branch>`: scaricare gli ultimi cambiamenti dal remoto;
- `git fetch`: scaricare i cambiamenti dal remoto senza fare merge;
- `git log -oneline`: visualizzare la cronologia compatta dei commit;

- `git diff`: visualizzare le modifiche non ancora staged;
- `git diff -staged`: visualizzare le modifiche staged ma non committate;
- `git merge <branch>`: fondere un branch nel branch corrente;
- `git branch -d <branch>`: eliminare un branch locale (sicuro: rifiuta se ha commit non mergiati);
- `git branch -D <branch>`: eliminare un branch locale forzando (anche se non mergiato);
- `git stash`: salvare temporaneamente modifiche non committate;
- `git stash pop`: ripristinare le modifiche salvate con `stash`.

2.6 Best practices

2.6.1 Linee guida sui commit

1. **Commit piccoli e atomici**: ogni commit dovrebbe rappresentare un singolo cambiamento logico. Evitare commit "tutto in uno" con decine di file modificati per scopi differenti;
2. **Messaggi descrittivi**: il messaggio del commit deve descrivere *cosa* è cambiato e *perché*, non *come*;
3. **Commit frequenti**: meglio molti commit piccoli che pochi giganti. È più facile rivedere, rollback-are, cherry-pickare;
4. **Non committare file generati**: cache, output di backtest, file di lock di virtualenv. Questi sono gestiti via `.gitignore`;
5. **Non committare credenziali**: API key, password, token devono essere in `.env` (escluso da git) o in variabili d'ambiente.

2.6.2 Linee guida sui branch

1. **Branch corti**: un branch dovrebbe vivere al massimo qualche giorno, idealmente meno. Branch che vivono settimane sono difficili da mergiare e creano conflitti;
2. **Un branch per issue**: non mescolare multiple feature in un singolo branch;
3. **Nomenclatura coerente**: seguire la convenzione `feature/<numero>-<descrizione>` o le sue varianti (`fix/`, `docs/`);
4. **Sincronizzazione regolare**: pull da `main` almeno una volta al giorno se il branch vive più di un giorno.

2.6.3 Linee guida sulle pull request

1. **PR piccole**: una PR dovrebbe modificare al massimo qualche centinaio di righe. PR enormi sono difficili da revisionare e tendono a contenere bug;
2. **Una PR per scopo**: non mescolare feature, fix e refactoring nella stessa PR;
3. **Descrizione chiara**: utilizzare il template descritto nella Sezione precedente;
4. **Test prima di chiedere review**: assicurarsi che il codice giri localmente e che i test passino *prima* di chiedere la review;
5. **Rispondere ai commenti**: rispondere a ogni commento di review, anche solo con un "fatto" o un emoji di conferma;
6. **Non auto-mergiare**: anche quando si è l'unico maintainer, è buona pratica aspettare almeno una review prima di mergiare.

2.6.4 Linee guida sul codice

1. **PEP 8**: seguire le convenzioni di stile Python standard;
2. **Docstring**: ogni funzione pubblica e classe deve avere una docstring che descriva scopo, argomenti, valori di ritorno;
3. **Type hints**: utilizzare type hint dove appropriato per migliorare la leggibilità;
4. **Niente codice morto**: rimuovere codice commentato. Se serve come riferimento, è già nella storia git;
5. **Niente `print()` in production**: usare il modulo `logging` per output diagnostico.

2.6.5 Anti-pattern da evitare

- **Forzare push su branch condivisi**: `git push -force` su branch su cui lavorano altre persone causa perdita di lavoro. Usare `-force-with-lease` se proprio necessario, e comunicare al team in anticipo;
- **Commit di binari grandi**: parquet, CSV grandi, modelli ML pesanti non vanno committati. Usare la cache locale o Git LFS se davvero necessario;
- **Branch nominati male**: branch chiamati `test`, `nuova`, `mio` non aiutano nessuno;
- **Messaggi di commit vuoti**: `git commit -m "wip"` o `"fix"` senza altre informazioni rendono la storia inutile.

Capitolo 3 Sviluppo di strategie di trading

3.1 Obiettivo

Questo capitolo definisce la procedura completa per lo sviluppo di una nuova strategia di trading nel framework QuantSys Institutional, dalla **lettura della issue GitHub** che richiede la strategia fino alla sua **conclusione**, che può avere due esiti:

- **Esito positivo**: la strategia passa la validazione walk-forward OOS e viene integrata nel sistema tramite pull request;
- **Esito negativo**: la strategia non passa la validazione e viene archiviata insieme a un documento `_FAILED.md` che ne registra le cattive performance, in modo che le scelte fatte siano tracciate ma il codice non venga più caricato dal sistema di discovery.

La procedura riportata in questo capitolo è specifica per lo sviluppo di strategie. Per la procedura generale di sviluppo (issue, branch, pull request) si veda il Capitolo 2.

3.1.1 Razionale della procedura

Il framework QuantSys è progettato per ospitare un insieme di strategie atomiche decorrelate. Ogni strategia rappresenta una scommessa specifica: una ipotesi di edge documentabile in letteratura o in dati storici. La procedura di sviluppo riflette questa filosofia: non si integra una strategia perché "funziona sul backtest passato", ma perché passa una validazione out-of-sample rigorosa.

L'esito negativo non è considerato un fallimento del processo: anzi, è informazione preziosa. Tenere traccia dei tentativi falliti tramite il file `_FAILED.md` previene il rischio di ri-implementare strategie che in passato si sono già rivelate non profittevoli, e fornisce un audit trail trasparente delle decisioni di design.

3.2 Dalla lettura della issue alla creazione del branch

3.2.1 Step 1: Identificazione della issue

Lo sviluppatore identifica sulla repository GitHub (https://github.com/Idrorepellente/quantsys_institutional) una issue etichettata con la label `strategy`, che richiede l'implementazione di una nuova strategia.

Una issue di richiesta strategia ben formata contiene:

- **Titolo**: nome della strategia (es. *Implementare Statistical Arbitrage Cointegration su S&P 100*);
- **Razionale**: motivazione economica e/o letteratura di riferimento;
- **Edge atteso**: Sharpe target unleveraged, decorrelazione attesa rispetto al resto del sistema;
- **Criteri di accettazione**: soglia di Sharpe OOS, MaxDD massimo, eventuale finestra temporale di validazione richiesta;
- **Riferimenti**: paper accademici, articoli, discussioni precedenti.

3.2.2 Step 2: Lettura della documentazione tecnica

Prima di iniziare lo sviluppo, è obbligatorio leggere o rileggere il documento `GUIDE_STRATEGIE_CUSTOM.m` nella radice della repository. Tale documento descrive:

- Struttura attesa della cartella della strategia;
- Campi obbligatori e opzionali del `StrategyDescriptor`;
- Contract minimo che l'implementation deve rispettare;
- Sistema di indicator pool condiviso;
- Sistema di exit policy (TP/SL);
- Procedura di test e walk-forward;
- Esempi reali delle strategie già implementate.

3.2.3 Step 3: Aggiornamento del branch `main` locale

Posizionarsi sul branch `main` locale e aggiornarlo con le ultime modifiche dal remoto:

```
git checkout main
git pull origin main
```

3.2.4 Step 4: Creazione del branch dedicato

La convenzione per i branch di strategia è:

```
git checkout -b feature/<nome-strategia>
```

dove `<nome-strategia>` è uguale al nome della cartella che ospiterà la strategia all'interno di `strategies/`. Esempio: per implementare la strategia `stat_arb_coint`:

```
git checkout -b feature/stat-arb-coint
```

Si noti che, per convenzione, il nome del branch usa il trattino come separatore mentre il nome della cartella usa l'underscore.

3.3 Implementazione della strategia

3.3.1 Step 5: Creazione della cartella della strategia

All'interno di `strategies/` creare una nuova cartella con il nome scelto:

```
mkdir strategies/stat_arb_coint
```

La cartella deve contenere almeno i seguenti tre file:

- `__init__.py`: rende la cartella un package Python ed espone il `DESCRIPTOR`;
- `strategy.py`: definisce il `DESCRIPTOR` (entry point per il discovery);
- Un file di implementation, ad esempio `cointegration.py`: contiene la classe Python che implementa la logica della strategia.

Tutti i dettagli sul contenuto di questi file sono nel documento `GUIDE_STRATEGIE_CUSTOM.md`.

3.3.2 Step 6: Implementazione del codice

L'implementazione della strategia segue il *contract* dell'interfaccia `AtomicStrategy` (definita in `strategies/base.py`). Le metodi e proprietà obbligatori sono:

- `style`: lo `StrategyStyle` che la strategia implementa (es. `STAT_ARB`);
- `allowed_asset_classes`: insieme di `AssetClass` che la strategia opera (es. `{EQUITY_STOCK}`);
- `generate_signals(t_decision, t_execution, indicators_by_symbol, portfolio)`: ritorna la lista dei `Signal` per la barra corrente.

Si raccomanda di:

- Mantenere la logica della strategia *stateless* dove possibile, accettando lo stato corrente del portfolio come parametro;
- Evitare lookahead bias indicizzando gli indicators sempre con `.loc[:t_decision]`;
- Riutilizzare gli indicators del pool condiviso (`required_indicators` nel descriptor) anziché ricomputarli internamente.

3.3.3 Step 7: Configurazione del DESCRIPTOR

Il `DESCRIPTOR` è il punto di ingresso del sistema di discovery. Esso dichiara:

- Nome, descrizione, categoria, timeframe della strategia;
- Factory function per istanziarla con parametri configurabili;
- `default_params`: parametri default tunabili via YAML;
- `required_indicators`: indicatori da pre-computare nel pool;
- `excluded_instruments` e `included_instruments`: per restringere l'universe;
- `exit_policy_type`: `signal_only`, `atr`, `custom`, o `combined`;
- `enabled`: `True` per attivare il discovery, `False` per disattivare temporaneamente.

3.3.4 Step 8: Creazione dello script walk-forward

In parallelo all'implementation della strategia, creare nella cartella `tests/` uno script di walk-forward dedicato. Convenzione: `tests/walk_forward_<nome_strategia>.py`.

Lo script deve:

- Caricare i dati dal cache appropriato (D1, M5, FX);
- Costruire l'engine di backtest con la configurazione del YAML;
- Istanziare la strategia (con `s.descriptor = STRATEGY_DESCRIPTOR` per consentire la discovery della exit policy);
- Lanciare il walk-forward rolling con finestre train/test definite;
- Salvare l'output (PnL, equity, metriche) in `output/strategies/`.

3.3.5 Step 9: Commit incrementali

Durante lo sviluppo, effettuare commit incrementali con messaggi chiari:

```
git add strategies/stat_arb_coint/
git commit -m "Aggiungi scheletro strategia stat_arb_coint"

git add strategies/stat_arb_coint/cointegration.py
git commit -m "Implementa Engle-Granger test su pair S&P 100"

git add tests/walk_forward_stat_arb_coint.py
git commit -m "Aggiungi script walk-forward per stat_arb_coint"
```

Effettuare il push del branch sul remoto in modo periodico per non perdere lavoro:

```
git push origin feature/stat-arb-coint
```

3.4 Validazione walk-forward

3.4.1 Step 10: Lancio del walk-forward

Una volta che la strategia è implementata e lo script di walk-forward è pronto, eseguire il backtest:

```
python tests/walk_forward_<nome_strategia>.py
```

oppure, in alternativa, lanciare il backtest dalla dashboard (tab *Run Backtest*).

L'esecuzione produce:

- File JSON con riepilogo metriche in `output/strategies/`;
- File parquet con equity OOS aggregata in `output/strategies/`;
- Log di esecuzione con eventuali warning.

3.4.2 Step 11: Analisi dei risultati

I risultati del walk-forward devono essere analizzati *onestamente*, confrontandoli con i criteri di accettazione dichiarati nella issue originale.

Le metriche chiave da considerare sono:

- **Sharpe ratio netto OOS**: il valore più importante. Una strategia individuale che non raggiunge almeno Sharpe 0.4-0.5 difficilmente porta valore all'ensemble;
- **CAGR netto OOS**: confrontato con il benchmark (SPY buy-and-hold);

- **Alpha vs benchmark**: il return non spiegato dal beta di mercato;
- **MaxDD**: stabilità della curva, no drawdown catastrofici;
- **Correlazione con stream esistenti**: se la nuova strategia ha correlazione superiore a 0.6 con uno stream già presente, il suo contributo marginale all'ensemble è limitato;
- **Comportamento in periodi di stress**: 2008, 2020, 2022. La strategia non deve crollare drasticamente in regimi avversi;
- **Stabilità nel tempo**: confrontare lo Sharpe nei primi anni del backtest con gli ultimi anni. Una strategia il cui edge si erode significativamente è un campanello d'allarme.

3.4.3 Step 12: Decisione di accettazione o scarto

A questo punto si presenta un bivio:

- Se la strategia rispetta i criteri di accettazione della issue, si procede con l'**esito positivo** descritto nella Sezione 3.5;
- Se la strategia non rispetta i criteri (Sharpe troppo basso, MaxDD eccessivo, instabilità inter-temporale, correlazione troppo alta con altri stream, ecc.), si procede con l'**esito negativo** descritto nella Sezione 3.6.

Anti-pattern: salvare a tutti i costi

In presenza di risultati insoddisfacenti, è tentante "aggiustare" la strategia con micro-ottimizzazioni ai parametri fino a ottenere numeri presentabili. Questo è curve-fitting e va evitato. Le iterazioni accettabili sono:

- Correggere bug nell'implementation (es. lookahead bias, errata gestione di dividendi);
- Aggiungere features mancanti richieste dalla logica originale della strategia (es. filtri di liquidità);
- Cambiare l'asset class su cui opera la strategia se il razionale lo giustifica.

Le iterazioni non accettabili sono:

- Cambiare lookback finché il Sharpe migliora;
- Escludere asset post-hoc perché hanno avuto performance peggiori;
- Aggiungere filtri di regime tunati sui dati di test;

- Restringere il periodo di backtest a quello in cui i numeri sembrano migliori.

Se ci si trova a iterare in queste direzioni, la strategia probabilmente non ha un edge robusto. Procedere all'esito negativo è preferibile a integrare nel sistema una strategia overfittata.

3.5 Esito positivo: integrazione tramite Pull Request

Quando la strategia passa la validazione walk-forward e rispetta i criteri di accettazione della issue, si procede con la sua integrazione nel sistema tramite pull request.

3.5.1 Step 13a: Aggiornamento documentazione

Prima di aprire la pull request, aggiornare i documenti che descrivono lo stato del sistema:

- Creare (o aggiornare) il file `README.md` all'interno della cartella della strategia (`strategies/<nome_strategia>/README.md`). Tale file deve contenere:
  - Razionale economico della strategia;
  - Riferimenti a letteratura;
  - Parametri default e loro motivazione;
  - Risultati OOS sintetici (Sharpe, CAGR, MaxDD, periodo coperto);
  - Eventuali limiti noti.
- Se la strategia introduce nuovi indicatori, aggiornare la sezione corrispondente di `GUIDE_STRATEGIE_CUSTOM.md`;
- Se la strategia modifica numeri rilevanti dell'ensemble complessivo, aggiornare `PITCH_FOF.md` con i nuovi valori.

3.5.2 Step 14a: Pull request finale

Aprire la pull request da `feature/<nome-strategia>` verso `main` seguendo il template descritto nella Sezione 2. La descrizione deve includere obbligatoriamente:

```
Closes #<numero-issue>

## Strategia: <nome>

## Descrizione
<descrizione sintetica del razionale economico>
```

```
## Risultati walk-forward OOS
- Sharpe netto: X.XX (target era >Y.YY)
- CAGR netto: X.XX% (benchmark SPY: Y.YY%)
- Alpha vs SPY: +X.XX%
- MaxDD: -X.XX%
- Periodo coperto: YYYY-MM-DD -> YYYY-MM-DD

## Correlazione con stream esistenti
- vs momentum:  X.XX
- vs quality_lv: X.XX
- vs value:     X.XX
- vs bear_overlay: X.XX

## Test eseguiti
- [x] Walk-forward completo
- [x] Stress test 2008 / 2020 / 2022
- [x] Verifica decorrelazione vs fattori standard
- [x] Test unitari sull'implementation

## Note
<eventuali note: dipendenze nuove, exit policy scelta,
limiti di capacity osservati>
```

3.5.3 Step 15a: Review e merge

La pull request entra nel normale ciclo di review descritto nel Capitolo 2:

1. Almeno un membro del team effettua la review;
2. Particolare attenzione viene prestata a: presenza di lookahead bias, qualità dei test, ripetibilità del walk-forward, plausibilità dei numeri OOS;
3. Eventuali modifiche richieste vengono implementate dall'autore;
4. Una volta approvata, la PR viene mergiata con strategia *Squash and merge*;
5. La issue collegata viene chiusa automaticamente.

A questo punto la strategia è parte del sistema. Verrà discoverata automaticamente dal registry e potrà essere inclusa negli ensemble.

3.5.4 Step 16a: Cleanup

Dopo il merge:

1. Cancellare il branch remoto da GitHub (l'interfaccia lo propone automaticamente);
2. Cancellare il branch locale:

```
git checkout main
git pull origin main
git branch -d feature/<nome-strategia>
```

3. Verificare che la strategia appaia nella dashboard come stream selezionabile.

3.6 Esito negativo: archiviazione con disabilitazione del discovery

Quando la strategia non rispetta i criteri di accettazione della issue, non viene integrata nel sistema ma il lavoro svolto **non viene cancellato**. Viene invece archiviato in modo che:

- Il sistema di discovery non carichi la strategia (per evitare di includerla involontariamente in qualche ensemble);
- Il codice resti nella repository come riferimento storico;
- Le ragioni del fallimento siano documentate, in modo che in futuro non si re-implementi la stessa strategia partendo da zero.

3.6.1 Convenzione del prefisso underscore

La convenzione adottata è la seguente: **le cartelle e i file il cui nome inizia con il carattere underscore _ vengono ignorati dal sistema di discovery delle strategie.**

In pratica, basta rinominare la cartella della strategia aggiungendo un _ all'inizio del nome per disattivarla dal sistema, lasciando però il contenuto consultabile.

3.6.2 Step 13b: Rinomina della cartella della strategia

Rinominare la cartella aggiungendo l'underscore davanti al nome:

```
git mv strategies/stat_arb_coint strategies/_stat_arb_coint
```

Il sistema di discovery, che scansiona `strategies/` cercando cartelle con `strategy.py` al loro interno, ignorerà automaticamente quelle che iniziano con `_`.

3.6.3 Step 14b: Creazione del file `_FAILED.md`

All'interno della cartella rinominata, creare un file `_FAILED.md` (anch'esso col prefisso `_` per esplicitarne lo stato) che documenta il fallimento. Il file deve contenere:

- Razionale originale della strategia (recuperato dalla issue);
- Risultati walk-forward OOS effettivamente ottenuti;
- Analisi del motivo del fallimento (interpretabile, non solo numerico);
- Eventuali iterazioni tentate e perché non hanno portato a un esito positivo;
- Raccomandazioni per il futuro (es. "non ri-provare questa stessa strategia su questo universe", oppure "ri-tentare se cambia la disponibilità di dati X").

Si raccomanda il seguente template:

```
# Strategia <nome>: archiviata per cattive performance

**Data archiviazione**: YYYY-MM-DD
**Issue di origine**: #<numero>
**Branch di sviluppo**: feature/<nome-strategia>

## Razionale originale
<descrizione dell'edge atteso, con riferimenti a letteratura>

## Criteri di accettazione (dalla issue)
- Sharpe netto OOS > 0.5
- MaxDD < 15%
- Correlazione con momentum < 0.3
- ...

## Risultati walk-forward effettivi
- Sharpe netto OOS: 0.18 (target era >0.5)
- CAGR netto: 2.4% (benchmark SPY: 10.2%)
- Alpha vs SPY: -1.8%
- MaxDD: -22.5%
- Periodo coperto: YYYY-MM-DD -> YYYY-MM-DD

## Analisi del fallimento
<spiegazione interpretabile del perche' la strategia non funziona.
Esempi tipici:
- L'edge in letteratura era documentato su dati pre-2010, post-2010
```

```
  e' stato eroso da decimalization e HFT.
- La correlazione con il momentum esistente e' 0.85, quindi non
  porta diversificazione.
- La strategia ha funzionato bene su 2018-2021 ma e' collassata
  nel 2022 (regime di rising rates) suggerendo dipendenza non
  modellata dai tassi.
- Il backtest gross-of-fees era promettente ma i costi di
  transazione hanno mangiato l'edge.>

## Iterazioni tentate
1. <descrizione iterazione 1 e perche' non ha funzionato>
2. <descrizione iterazione 2 e perche' non ha funzionato>
3. ...

## Raccomandazioni
- <indicazione su cosa NON ri-provare>
- <indicazione su cosa potrebbe valere la pena ri-tentare in futuro,
  e a quali condizioni>
```

3.6.4 Step 15b: Aggiornamento documentazione globale

Per dare visibilita' a livello di progetto del tentativo fallito, si raccomanda di:

- Aggiungere una riga nel registro globale dei tentativi (se presente, ad esempio in `ROADMAP_FASE2_v2.md` o in un eventuale `JOURNAL.md`);
- Aggiornare la issue di origine con un commento di chiusura che riassume l'esito e linka al `_FAILED.md` appena creato.

3.6.5 Step 16b: Pull request di archiviazione

A questo punto si apre una pull request che:

- Rinomina la cartella della strategia (commit con `git mv`);
- Aggiunge il `_FAILED.md` all'interno;
- Eventualmente aggiorna documentazione globale.

Il template di descrizione della PR di archiviazione e':

```
Closes #<numero-issue>

## Strategia: <nome> [ARCHIVIATA]
```

```
## Esito
La strategia non ha superato la validazione walk-forward.
Archiviata in `strategies/_<nome>/` con `_FAILED.md` che
documenta i risultati e le ragioni del fallimento.

## Numeri principali
- Sharpe netto OOS: X.XX (target era >Y.YY)
- CAGR netto: X.XX% (benchmark SPY: Y.YY%)
- MaxDD: -X.XX% (target era < Y.YY%)

## Motivo principale del fallimento
<una riga sintetica>

## Note
Il codice resta nella repository per riferimento storico.
Il sistema di discovery ignora automaticamente la cartella
grazie al prefisso `_`.
```

Anche questa PR passa per la normale procedura di review e merge. La differenza rispetto a una PR di esito positivo è semplicemente che il codice mergiato è "archiviato" anziché "attivo".

3.6.6 Step 17b: Cleanup

Identico allo Step 16a: dopo il merge, cancellare il branch remoto e locale.

3.6.7 Note operative sul `_FAILED.md`

1. Il file deve essere **onesto**: registrare i numeri effettivi, non versioni "rimaneggiate" per salvare la faccia;
2. Deve essere **interpretabile**: un altro sviluppatore che lo legga tra sei mesi deve capire perchè la strategia non ha funzionato;
3. Deve essere **actionable**: deve aiutare a non ripetere lo stesso errore. Se il problema era nei dati, indicarlo. Se era nell'edge atteso, indicarlo;
4. Non e' un *post-mortem* burocratico: l'obiettivo e' costruire memoria collettiva del progetto, non riempire un campo.

3.7 Quadro riassuntivo

In sintesi, il ciclo di vita di una strategia all'interno del progetto QuantSys Institutional segue il seguente schema, partendo dalla lettura della issue:

1. **Issue GitHub** richiede una nuova strategia con criteri di accettazione espliciti;
2. Lo sviluppatore **legge la documentazione tecnica** (`GUIDE_STRATEGIE_CUSTOM.md`);
3. Crea il **branch dedicato** `feature/<nome-strategia>` a partire da `main` aggiornato;
4. **Implementa** la strategia come plugin in `strategies/<nome_strategia>/`;
5. Crea lo **script di walk-forward** in `tests/walk_forward_<nome>.py`;
6. Effettua **commit incrementali** e push periodici;
7. Lancia il **walk-forward OOS** e analizza i risultati;
8. In base ai risultati si presentano **due possibili esiti**:
    - **Esito positivo**: strategia valida la performance attesa → pull request di integrazione con aggiornamento documentazione e numeri OOS in descrizione → review → merge → strategia attiva nel sistema;
    - **Esito negativo**: strategia non rispetta i criteri → rinomina cartella aggiungendo prefisso `_` → creazione di `_FAILED.md` che documenta cattive performance e ragioni del fallimento → pull request di archiviazione → merge → codice conservato come riferimento ma ignorato dal discovery.

In entrambi gli esiti il lavoro è tracciato, riproducibile e auditable. Questo è coerente con la filosofia del progetto: ogni decisione (sia di integrazione che di scarto) deve essere documentata e basata su evidenza empirica, non su intuizione o su numeri di backtest non validati.

Capitolo 4 Verbali e meeting

4.1 Obiettivo

Questa sezione descrive il processo di generazione e archiviazione dei verbali dei meeting di progetto. Lo scopo è garantire che le decisioni prese durante gli incontri siano tracciate in modo trasparente e consultabili in futuro dai membri del team.

4.2 Struttura del verbale

Ogni verbale deve contenere le seguenti sezioni:

- **Data e orario**: data del meeting e durata;
- **Partecipanti**: nomi dei presenti e degli eventuali assenti giustificati;
- **Argomenti**: lista degli argomenti trattati durante il meeting;
- **Decisioni**: decisioni operative prese, con esplicito riferimento a issue o pull request quando appropriato;
- **Azioni**: azioni da intraprendere a valle del meeting, ciascuna con un responsabile e una scadenza;
- **Punti aperti**: questioni discusse ma non risolte, da riprendere in meeting successivi.

I verbali sono salvati nella cartella `PB/verbali/` con nome `verbale_DD_MM_YYYY.tex`.

4.3 Procedura

1. Durante il meeting, un membro del team designato prende appunti;
2. Al termine del meeting, gli appunti vengono strutturati nel formato descritto nella Sezione precedente;
3. Il verbale viene scritto in LaTeX nella cartella `PB/verbali/`;

4. Viene aperto un branch `docs/verbale-YYYY-MM-DD` contenente solo il nuovo verbale;
5. Viene aperta una pull request che viene mergiata dopo verifica da parte di un altro partecipante al meeting (per garantire l'accuratezza del contenuto);
6. Il verbale è quindi disponibile a tutto il team.

4.3.1 Frequenza

Si raccomanda di redigere un verbale per:

- Ogni meeting di pianificazione settimanale;
- Ogni decisione strategica significativa (es. approvazione di una nuova strategia per l'ensemble core);
- Ogni meeting con stakeholder esterni (es. presentazione preliminare a Fund of Funds).

Riunioni puramente operative di breve durata (es. stand-up giornalieri) possono essere documentate in modo più sintetico o via messaggistica del team.